

* Data vs Information

- Data: facts concerning objects and events that can be recorded & stored on computer media
- Information: Data that has been processed to increase knowledge of person using it.

* Data is stored in data base and it is processed into information to give it more meaning.

* Manual filing systems

- Traditional method of organizing records where a manual files are made.
- Files are labeled and kept in locked cabinets to ensure security.
- Only Feasible when data is small or ~~base~~ only basic storage and retrieval is needed.
- Limitations of Manual Filing Systems
 1. Sequential Data Retrieval: Searches begin from first entry to last
 2. Limited Processing of data: Complex queries that require data from multiple different sources.
 3. Scaling Challenges: Difficult to manage for large data

* File-based systems

- Predecessors of modern databases, they were an initial attempt to computerize traditional manual filing systems.
- Application - Centric Design: It is a collection of Application Programs that perform services for end users.
- Decentralized Approach: The systems were decentralized, meaning each department had its own independent system to handle its specific data.
- Data Management: No centralized authority rather each application program is fully responsible for its own data - defining it, storing it, and retrieving it.

Signature _____

RC

No. _____

- Limitations of File Based Approach

1. Separation and Isolation of Data: Each program maintains its own data, users of one program may be unaware of useful data in another
2. Data Duplication and Redundancy: Storing the same data multiple times leads to waste of space and changes are lead to inconsistency
3. Requires Extensive Programming in third generation: time consuming
4. Data dependence: File structure is defined within the program, so any changes in the characteristics require changes in access programs
5. Incompatible File Formats: Programs were often written in different languages, so they couldn't easily access each other's files
6. Fixed Queries: Programs are written for specific functions, with new requirements needing a new program
7. Limited data Sharing: Due to data being decentralized

★ Database

- A ~~any database~~ is an organized collection of related data, and a description of the data, designed to meet information needs of an organization.
- Relational Database is a collection of tables linked together by common fields.

★ Database

- A database is an organized collection of data designed to meet information needs of an organization.

Includes:

- Organization of data
- Efficient Retrieval of data
- Reliable storage of data
- Maintaining data consistency
- Sharing data
- Structuring the data
- A relational database is a collection of tables linked together by common fields

- Database Management System (DBMS) is the software that allows you to define, create, maintain, and control access to the database.
- These functionalities are achieved by support of database languages:
 1. The Data Definition Language (DDL) is used to define the database by specifying data types, structures, and constraints on the data.
 2. The Data Manipulation Language (DML) is used to manage the data itself, allowing users to insert, update, and retrieve the data.
 3. The Data Control Language (DCL) is used to control who can access what in the database.
 4. The Transaction Control Language (TCL) is used to control transactions ensuring data consistency.

- Database System is the combination of DBMS software and data.
- Database Application is a program that interacts with the database to perform specific tasks.
- Schema is the formal structure of the database, containing its tables & attributes.
- Metadata is data about data, it provides descriptions of data being stored within the system.

★ Components of a DBMS environment

1. Hardware: The physical devices required to run the DBMS and application. (single computer, network of computers, or a mainframe)
2. Software: Includes DBMS software, operating system, network software and application programs.
3. Data: Most important as it acts as a bridge between machine and human components. Contains both operational and metadata.
4. Procedures: Documented instructions and rules that govern how to use and manage the system.

Signature _____

Date _____

No. _____

④

5. People: Individuals involved in the system.

* Roles in the Database Environment.

1. Database Administrator (DA):

- DA is a planning and policy role
- They decide:
 - ^{defining} Data policies, standards & rules.
 - What data should be stored & how it should be used.
 - Enforcing security and privacy rules

2. Database Administrator (DBA)

- They take policies & decisions of the DA & implement them
- They install and configure the DBMS software.
- Creates the actual databases and maintains them
- Controls user access & security
- Performs backup & recovery
- Monitors performance & tunes the system

3. Database Designer

a) Logical Database Designer

- Designs logical structure of the database
 - Identifies entities, attributes and relationships.
 - Creates ER diagrams & schemas.
 - Normalizes to remove redundancy.

b) Physical Database Designers

- They decide how data is physically stored and optimized
- Their job is performance & storage efficiency
 - File Organization: How records are physically arranged on a disk
 - Indexing: to improve performance
 - Works with DBA to improve access speeds

4. Application Programmers

- Develop software that interacts with the database
- Write programs using SQL and programming languages.
- Implement business rules in application.

Signature _____

RC

No. _____

E. End Users

a) Native Users

- Users who use predefined applications
- Use forms and buttons not SQL
- Eg: Bank tellers, ATM users, clerks

b) Sophisticated Users

- Experts who directly use the DBMS
- Write complex SQL queries and perform data analysis
- Eg: Data Analysts, Engineers, researchers

Advantages	Disadvantages
• Data Consistency & Control of Data Redundancy • Data Independence • Improved Data Sharing & Integrity as data is centralized. • Enhanced security due to access control • Better accessibility due to easier & faster retrieval	• Complexity • Size • Cost of DBMS • Additional Hardware cost • Performance • Greater impact of failure as if DBMS goes down, everything goes down.

* 3 Level ANSI-SPARC Architecture

- Conceptual Framework for designing and organizing DBMS
- Separates database into three distinct levels, so that changes at one level don't break the others. This separation is called data independence

1. External Level

- What individual users or applications see.
- Each user gets their own personalized view of the data

Signature _____

RC

No. _____

2. Conceptual Level

- Complete logical picture of the entire database
- No single user sees the in full.
- It represents all entities, attributes, and relationships, Constraints and Security and Integrity Information

3. Internal Level

- Physical reality of the database
- It handles storage space allocation, record description, record placement, compression and encryption.

External Level → What data each user sees (view) (Multiple)

Conceptual Level → What data exists and how it connects

Internal Level → How data is stored, indexed, retrieved, compressed and encrypted

* Database Schema is the overall description of the database

* Database Sub-Schema is used to create different and personalized view of the same data.

* The data in the database at any point is called the database instance

* Schema is also called the state or extension of the database

* There are multiple external schemas but only one conceptual and internal schema.

* Data Independence

- Goal of 3 Level Architecture is to achieve data independence, which means upper levels are unaffected by changes to lower level.

1. Logical Data Independence: Immunity of the external schema to changes on the conceptual schema.

2. Physical Data Independence: The immunity of conceptual schema to changes in the internal schema.

Signature _____

RC

No. _____

External Schemas

↕ Logical Data Independence

Conceptual Schemas

↕ Physical Data Independence

Internal Schemas

* Database Languages

- Data Definition Language: Structured Query Language
- Data Manipulation Language
 - Procedural DML (give every step)
 - Non-Procedural DML
 - SQL
 - Query By Example (QBE)
- Fourth Generation Language
 - Form, report, graphics, & application generators

* Data Models

- A Data Model is a set of concepts used to define what data exists, how it relates to other data and what constraints apply to it
- 3 Types Exist:

1. Object-Based Data Models

- Focused on representing real world things and their relationship
- Used mostly in design and documentation phase
 - ER Model - most common, uses entities, attributes, and relationships to map out the database visually.
 - Object-Oriented Model - extends this idea for software development, treating data as objects with behavior attached.

2. Physical Data Models

- Describe how data is stored in the computer
- Deals with how the DBMS stores and accesses data under the hood

Signature _____

PC

No. _____

2.3. Record Based Data Model

- How data is actually logically structured inside the database.

a) Relational Data Model

- Most commonly used today
- Relation (Table), Attribute (Column), & Tuples (Row)
- Supports one-many relation

b) Network Model

- Obsolete
- Many-to-many relationship support
- Pointers are set between records
- Used for Large Volume transaction processing as it is fast due to hard coded pointers

c) Hierarchical Model

- Obsolete
- One-to-many Relationship
- Pointer between parent node and child node
- No many to many relationship
- data has tree like structure.

* Conceptual Modeling is the process of creating conceptual schema - capturing all entities, relationships, and constraints of the entire organizations data in a single complex logical picture - External Schema are then derived from the conceptual schema.

* Functions of a DBMS

- | | |
|------------------------------------|--|
| • Data storage, retrieval & update | • Integrity Services |
| • Transaction support | • Utility Services |
| • Concurrency control services | • Services to promote data independence. |
| • Recovery services | |
| • Authorization Services | |

Signature _____

RG

No. _____

* The Relational Model

~~data~~ I

- Objective:

- Achieving high degree of data independence.
- Improving data consistency and reducing redundancy through the use of normalization, which involves creation relations with no repeating groups.
- Set Oriented Data Manipulation Languages Support (like SQL)

- Structure

- Relation: A table with rows and columns.
- Attribute: A named column of a relation.
- Domain: Set of allowable values for one or more attributes.
- Tuple: A row of a relation.
- Degree: The number of attributes a relation contains.
- Cardinality: The number of columns a relation contains.
- Relational database: A collection of normalized relations with distinct relation names.

Relation $\rightarrow$ Table $\rightarrow$ File

Tuple $\rightarrow$ Row $\rightarrow$ Record

Attribute $\rightarrow$ Column $\rightarrow$ Field

- Mathematical Relations

- The Cartesian Product of two sets produces all possible ordered pairs. A relation is simply any subset of that product.

Cartesian Product $\rightarrow$ All possible row combinations

Subset $\rightarrow$ Actual Table

- Relation Schema

- Let $A_1, A_2, \dots, A_n$ be attributes with domains $D_1, D_2, \dots, D_n$
- The set $\{A_1:D_1, A_2:D_2, \dots, A_n:D_n\}$ is called a Relation Schema
- Relation (R) is a set of tuples that follow this schema.

Each tuple has the form:

$(A_1:d_1, A_2:d_2, A_3:d_3)$ where $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$
 $\downarrow$
 belongs to

* A relation is a subset of Cartesian product of attribute domains.

* Relational Database schema or Relational schema is a set of relation schema with distinct names.

$$R = \{R_1, R_2, \dots, R_n\}$$

where $R_1, R_2, \dots, R_n$ are set of relation schema

- Properties of Relations

- Name distinct from other relations in the relational schema.
- Each cell of the relation contains exactly one atomic value.
- Each attribute has a distinct name.
- Values of an attribute are all from same domain.
- Each tuple is distinct with no duplicates.
- Order of attributes has no significance.
- Order of tuples has no significance theoretically. (In practice, it can affect accessing efficiency).

- Relational Keys

- **Super Key:** Any attribute or set of attributes that uniquely identify a tuple.
- **Candidate Key:** A minimal superkey that uniquely identifies a tuple and removing any attribute breaks uniqueness.
- **Composite Key:** A candidate key with more than one attribute.
- **Primary Key:** The one candidate key chosen to officially identify tuples. There can only be one.
- **Alternate Key:** Candidate keys that weren't chosen as primary key.
- **Foreign Key:** An attribute in one table that references a candidate key in another table.

- Integrity Constraints

Rules that keep data accurate and consistent

- **NULL:** It means no value exists.
- **Entity Integrity:** No attribute of a primary key can be NULL.
- **Referential Integrity:** A foreign key must refer a candidate key in its referenced relation or be NULL.
- **General Constraints:** Custom business rules set by users or DBA.

- Views

- A **Base Relation** is a real physical table stored in the database.
- A **View** is a virtual table that doesn't exist. It is dynamically produced by running a query on base relations whenever the user requests it. Since it is generated each time, it reflects current state of base data.

• Purpose of Views:

- **Security:** Hide Sensitive Columns
- **Customization:** Each user sees data shaped for their role
- **Simplification:** Complex joins can be presented as a single clean table
- **External Schema Support:** Views are how external schemas are implemented.

Signature _____

Dr. _____

No. _____

• Updating Views

- Changes to base table automatically reflects in views but not vice versa.
- Views are defined between not updateable, partially updateable or fully updateable.

Updates allowed $\rightarrow$ View from single table with primary/candidate key

Updates not allowed $\rightarrow$ Views using joins (multiple tables)

Updates not allowed $\rightarrow$ Views using aggregation or Group By

* Relational Algebra and Relational Calculus

Relational Algebra = You tell it how to get data (Procedural)

Relational Calculus = You describe what you want (Non Procedural)

Relational Completeness = A language is relationally complete if it can produce anything the relational calculus can.

- * The output of relational algebra is a query relation, so you can chain operations making complex queries possible. This is called closure.

- * Selection and Projection are Unary Operations as they operate on one relation. Rest are Binary Operators

Fundamental Operations: Selection, Projection, Cartesian Product, Union, Set Difference

Derived Operations: Intersection, Division, All Joins

Day: _____

Date: _____

* Unary Operators

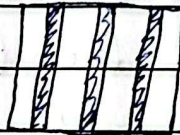
1) Selection (or Restriction)



- Used to filter rows based on a condition (predicate).
- $\sigma_{\text{predicate}}(R)$
- Works on a single relation R and as output gives a relation that only contains tuples from R that satisfy the condition

Eg: List all staff with a salary greater than 10000
 $\sigma_{\text{salary} > 10000}(\text{Staff})$

2) Projection



- Filters columns
- $\pi_{a_1, a_2, \dots, a_n}(R)$
- Where a_i are attributes / columns & R is a relation
- Works on a single relation R and defines a relation that contains a vertical subset of R , extracting values you want & eliminating duplicates

Eg: Produce a list of salaries for all staff, showing only the staff No, fName, lName, and salary details.

$\pi_{\text{staffNo}, \text{fName}, \text{lName}, \text{salary}}(\text{Staff})$

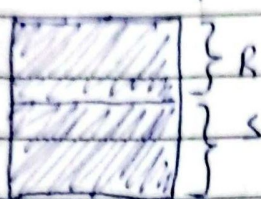
* Set Operations

Set Operations are binary operations used to combine information from several relations.

2) Union

$$R \cup S$$

- Combines all tuples from R and S, removing duplicates



- Requires Union Compatibility: For R and S to be union-compatible they must have same number of attributes (a) and matching domains in order (b)

(a) If R has 3 columns, S must also have 3.

(b) The 1st column of R and 1st column of S must hold same type of data (eg. strings, numbers). Same for 2nd, 3rd, and so on.

Eg: List all cities where ^{there is} either a branch office or a property for rent

$$\pi_{city}(Branch) \cup \pi_{city}(Property\ For\ Rent)$$

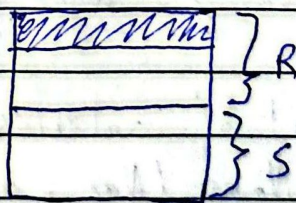
2) Set Difference

$$R - S$$

- Set difference defines a relation

consisting of tuples that are in R only and not S.

- R and S must be union compatible



Eg: List all cities where branch there is a branch office but no properties for rent

$$\pi_{cities}(Branch) - \pi_{cities}(Property\ For\ Rent)$$

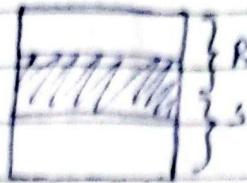
3) Intersection

- $R \cap S$

- Intersection operation

defines a relation consisting of tuples in both R and S .

- R and S must be union compatible.



Eg: List all the cities where there is both a branch office and at least one property for rent.

$$\pi_{city}(\text{Branch}) \cap \pi_{city}(\text{PropertyForRent})$$

- Intersection is a derived operation as it can be expressed using set difference.

$$R \cap S = R - (R - S)$$

4) Cartesian Product

- $R \times Q$

- Cartesian product multiplies

two relations to get a relation consisting of all possible pairs of tuples from both relations.

- The Cartesian Product Relation will contain $(I * J)$ tuples with $(N + M)$ attributes.

- If attributes have same names they are prefixed with relation name to maintain uniqueness.

Eg: client.ClientNo and Viewing.ClientNo

P	Q	$P \times Q$
A	B	A B
a	1	a 1
a	2	a 2
a	3	a 3
b	1	b 1
b	2	b 2
b	3	b 3

Eg: List the names and comments of all clients who have viewed a property for rent

$$(\pi_{\text{clientNo}, \text{fName}, \text{lName}}(\text{Client})) \times (\pi_{\text{clientNo}, \text{propertyNo}, \text{comments}}(\text{Viewing}))$$

↳ This still contains extra tuples

$$\sigma_{\text{client.ClientNo} = \text{Viewing.ClientNo}}((\pi_{\text{clientNo}, \text{fName}, \text{lName}}(\text{Client})) \times (\pi_{\text{clientNo}, \text{propertyNo}, \text{comments}}(\text{Viewing})))$$

↳ This now does what we want

- Decomposing Complex Operations (Used to split complex queries in small ones)

2) Assignment Operation ($\leftarrow$)

• Used to name the results of a relational algebra operation

$\text{TempViewing}(\text{clientNo}, f\text{Name}) \leftarrow \pi_{\text{clientNo}, f\text{Name}}(\text{Viewing})$

$\text{TempClient}(\text{clientNo}, l\text{Name}) \leftarrow \pi_{\text{clientNo}, l\text{Name}}(\text{Client})$

$\uparrow \text{Comment}(\text{clientNo}, l\text{Name}, v\text{clientName}, f\text{Name}) \leftarrow \text{TempViewing} \times \text{TempClient}$

2) Rename Operation (ρ)

$S1 \leftarrow \rho_s(\text{Staff})$

$S2 \leftarrow \rho_s(a_1, a_2, \dots, a_n)(\text{Staff})$

★ Join Operations

- Normally we only need combinations of cartesian products that satisfy a specific condition which is why we use joins.
- Joins are derivative operations of cartesian product

1) Theta Join (θ -Join)

• $R \bowtie_F S$

A	B
1	2
6	2

A	B
1	x
3	y

A	B	C
1	2	x
1	2	y

• The Theta Join defines a relation that contains tuples from cartesian product of R & S that satisfy the predicate F.

• The predicate can be $=, \neq, <, \leq, >, \geq$

$$R \bowtie_F S = \sigma_F(R \times S) \leftarrow \text{Derived Operation}$$

★ If predicate only contains $=$ the join is called Equijoin.

Eg: List the names & comments of all clients who have viewed a property for rent

$(\pi_{\text{clientNo}, f\text{Name}, l\text{Name}}(\text{Client})) \bowtie_{\text{clientNo} = \text{Viewing.clientNo}} (\pi_{\text{clientNo}, \text{comment}}(\text{View}))$

2) Natural Join ($R \bowtie S$)

T		U		T $\bowtie$ U		
A	B	B	C	A	B	C
a	1	1	x	a	1	x
b	2	3	y			

- Natural Join is an equijoin of two relations R and S over a common attribute x.

Eg: List the names & comments of all clients who have viewed a property for rent

$(\pi_{clientNo, fName, lName}(Client)) \bowtie (\pi_{clientNo, propertyNo, comments}(View))$

3) Outer Join

a) Left Outer Join ($\ltimes$)

- Keeps all rows from left relations with NULLs for unmatched right ones.

b) Right Outer Join ($\rtimes$)

- Keeps all rows from right relation, NULLs for unmatched left ones.

c) Full Outer Join ($\ltimes\rtimes$)

- Keeps all rows from both sides, NULLs where there's no match

T		U		T $\ltimes$ U			T $\rtimes$ U			T $\ltimes\rtimes$ U		
A	B	B	C	A	B	C	A	B	C	A	B	C
a	1	1	x	a	1	x	a	1	x	a	1	x
b	2	3	y	a	1	y	a	1	y	a	1	y
				b	2	Null	Null	3	y	Null	3	y
										b	2	Null

Left Outer Join

Right Outer Join

Full Outer Join

Eg: Produce a status report on property viewing. Showing property details and their views

$(\pi_{propertyNo, street, city}(PropertyForRent)) \ltimes\rtimes Viewlog$

* Natural Joins can be

* Outer Joins can be Theta Joins or Equi Natural Joins.

Day: _____

Date: _____

4) Semijoin

• $R \bowtie_r S$

T	
A	B
1	2

U	
B	C
1	2

T $\bowtie_r$ U	
A	B
a	1

- Semijoin returns rows from one side only, specifically the rows from the left relation that have at least one match in the right relation.
- It reduces the number of attributes and tuples in final relation saving resources

Eg: List complete details of staff that work at Glasgow
 $\text{Staff} \bowtie_{\text{staff Branch No} = \text{Branch Branch No} (\text{City} = \text{"Glasgow"} (\text{Branch}))}$

★ Division

• $R \div S$

- If X is an attribute of R and Y is an attribute of S .
 $R \div S$ answers which X is associated with all Y

Staff Course

S1 C1

S1 C2

S2 C1

S3 C2

S3 C2

Staff $\div$ Course

S1

~~S2~~ S3

* S2 is dropped as they have only completed C1

Eg: Identify all clients who have viewed all properties with 3 rooms.

$(\pi_{\text{clientNo}, \text{propertyNo}}(\text{Viewing})) \div (\pi_{\text{propertyNo}}(\sigma_{\text{rooms}=3}(\text{Property for Rent})))$

↳ Resulting Table will only have clientNo

★ The attribute that comes from S is removed (propertyNo in this case)

* Aggregation Operation

Main Aggregate functions are:

Count COUNT

SUM

AVG

MIN

MAX

• $AL(n)$

- where R is input Relation

- where AL is list of (aggregate function, attribute) pairs

Eg: COUNT(staffNo), AVG(salary) (staff)

Eg: How many properties cost more than \$350 per month to rent

$\rho_R(MyCount)$ Count property No (Rent > 350 (Property for Rent))

Eg: Find min, max, avg staff salary

$\rho_R(myMin, myMax, myAvg)$ MIN(salary), MAX(salary), AVG(salary) (staff)

* Grouping Operation

• Grouping first then aggregate per partition

• $GA AL(R)$

• GA is grouping attribute, AL is aggregation list, R is Relation

Eg: Find the Num of staff working at each branch and sum of their salaries

$\rho_R(branchNo, myCount, mySum)$ branchNo COUNT staffNo, SUM salary (staff)

*ER Diagram

- ER Modeling is top down where you start with entities and relationships then add attributes & constraints
- Enhanced Entity Relationship (EER) = ER + generalization + aggregation + composition

*Entity Types

- Entity type = the category / template = the table itself
- Entity occurrence = one specific instance of that category = one row in that table
- Entity = general term used loosely to mean either of the above depending on context

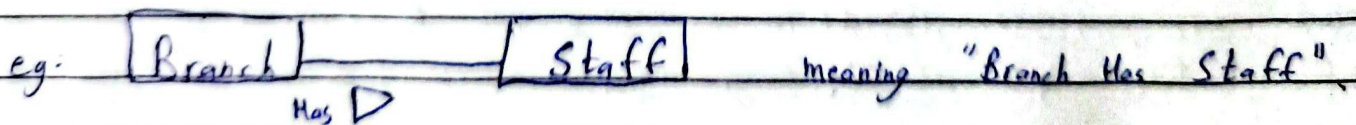
- In UML diagrams, entity types are drawn as rectangles,

*Representation Styles of ER-Diagram

- Chen Notation
- Crow's Foot Notation
- UML Notation

*Relationship Types

- Relationship type = category of association between entity types
- Relationship occurrence = an specific association between specific entity occurrences
- Relationships are shown through a line connecting two entity rectangles
- Named with a verb phrase. eg: Has, Manages, leased by
- Named in one direction only with an arrow showing which way to read it.



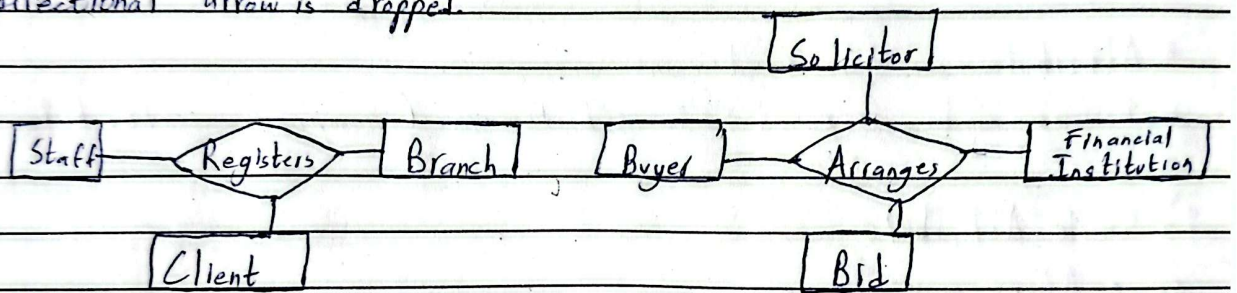
Day: _____

Date: _____

* Degree of Relationship Type

- Degree is the number of entity types participating in a relationship.
- A relationship of degree two is called Binary.
- A relationship of degree three is called Ternary.
- A relationship of degree four is called quaternary.
- A relationship of degree higher than binary is called complex relationship.

* Diagrammatic Rule: Binary relationships use a plain line, but ternary and above use a diamond shape with the relationship name inside, and the directional arrow is dropped.



Ternary Relationship

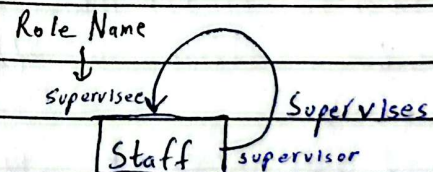
"Staff registers client at Branch"

Quaternary Relationship

"A solicitor arranges a bid on behalf of a buyer supported by a financial institution"

* Recursive Relationship

- In Recursive relationships one entity type participates in the same relationship, with different roles.



"Staff (supervisor) supervises Staff (supervisee)"

- Role Names

- Required for recursive relationships.
- Used when two entities have multiple relationships.

Day: _____

Date: _____



"Manager Manages Staff"

"Manager manages Staff Office"

"Branch office has member of staff staff"

* Attributes

- Attribute = A property of an entity or a relationship type
- Attribute Domain = The set of allowable values for one or more attributes.

* Attributes can share a domain

* Domains can be composed of sub domains (composite attributes)

1) Simple Attribute

- Atomic
- Cannot be broken down further.
- Single Component eg: salary, position

2) Composite Attribute

- Can be broken into smaller independent parts
- Eg: address → street + city

* The decision to make address simple vs composite depends on whether the system needs to use parts individually.

Simple Attribute → If you'll only print the full address.

Composite Attribute → If you need to search by city.

3) Single Valued Attributes

- An entity attribute that holds one value per entity occurrence.
- Majority of Attributes are single valued.

4) Multi Valued Attribute

- An attribute that holds multiple values per entity occurrence.
- Eg: phoneNo - a person can have home, work, and many more numbers.
- They can have upper and lower limits on quantity.

5) Derived Attributes

- An attribute that is derivable from the value of a related attribute or set of attributes, not necessarily in the same entity type.
- Eg: Age derived from date Of Birth

★ Keys

1) Candidate Keys

- Minimal Set of attributes that uniquely identify each entity occurrence.
- Cannot contain NULLs.

2) Primary Keys

- The candidate key you select as the main identifier.
- Selection Criteria:
 - Shorter length preferred (staff No over CNIC)
 - Fewer attributes preferred
 - Future uniqueness must be guaranteed.

3) Alternate Key

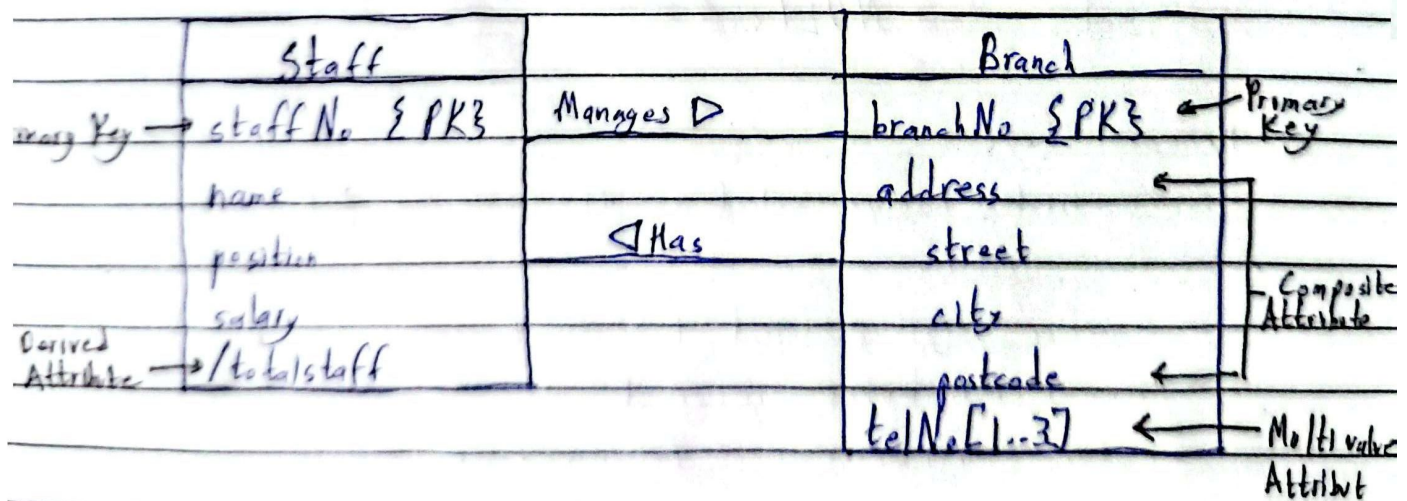
- A Candidate Key that wasn't chosen as primary Key.
- eg: CNIC when staffNo is primary Key.

4) Composite Key

- Made up of multiple attributes, whose values are unique together but not individually.

- Diagrammatic Representation

- * If an entity is to be displayed with its attributes we divide the entity's rectangle in two.
- * Primary Key is the first attribute represented by $\{PK\}$
- * attribute names start with lower case (Camel Case) (myVariable Name)
- * Alternate Key by $\{AK\}$
- * Derived Attribute by $\{ \}$ /
- * Composite Attribute by Indentation
- * Multivalued Attribute have range labeled with $[1..3]$ or $[1..*]$
- * When your primary Key is composite, each individual attribute that forms part of that composite key is tagged $\{PPK\}$ where PPK means Partial Primary Key



* Strong and Weak Entity Types

1) Strong Entity Type

- An entity that is not existence dependent on another entity.
- Each entity occurrence is uniquely identifiable using primary Key attributes of that entity type.
- Parent, Owner, Dominant

2) Weak Entity Type

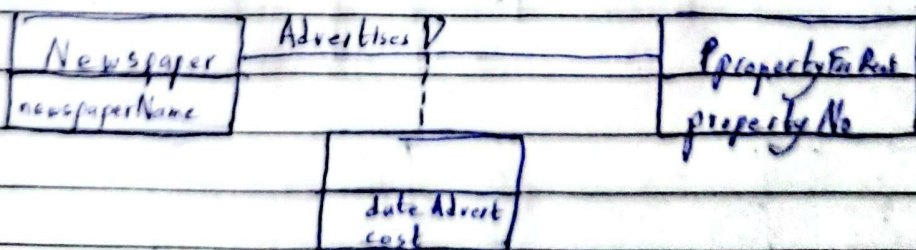
- An entity that is existence dependent on some other entity.
- Child, Dependent, Subordinate
- Has no primary Key of its own and depends on foreign Key from owner entity to be identified. (PK of others)

Eg: A Preferences record makes no sense without a Client to whom!

* Attributes on Relationships

- Relationships can have their own attributes when data belongs to the association itself rather than the entity.

- * Relationship Attributes are shown in a rectangle (same as entity) but connected to the relationship line with a dashed line.



* **Multiplicity** = how many occurrences of one entity can relate to occurrences of another entity through a relationship.

2) One-to-One Relationship (1:1)

General format: min..max

- 0 = optional (can have zero)
- 1 = mandatory (must have atleast one)
- * = many (no upper limit)

Staff	Manages	Branch
staffNo	1..1	branchNo

- "Staff manages zero or one branch"
- "Each branch is managed by only one staff"

* One to One Relationship refers to only maximum on both sides being 1
 * The number you place next to an entity describes how many times that entity can appear in the relationship

2) One-to-Many Relationship (1:*)

1:* means one side can have many, other side has at most one.

Newspaper	Advertises	Property For Rent
newspaperName	0..*	propertyNo

Staff	Oversees	Property For Rent
staff No	0..1	propertyNo

- "Each property is overseen by zero or one staff"
- "Each staff manages or oversees zero or more properties"

Date: _____

u) Many-to-Many Relationships (*:*)

- * *: * means both sides can have many occurrences related to each other

* In ER diagrams *: * relationships get broken down into 1: * relationships during conversion to a relational schema by introducing a junction/bridge table.

Newspaper	Advertises ▷	Property For Rent
newspaperName	0..*	1..* propertyNo

- * "Each property for rent is advertised in zero or more newspapers"

u) Multiplicity for Complex Relationships

- * In Binary ^{relationship} multiplicity you ask how many occurrences of B for one A.
- * In Ternary relationship you fix 2 entities and ask how many of third can exist.

Eg: You pick staff and branch to ask how many clients can a specific staff register at a specific branch.

- * For an n-ary relationship, fix (n-1) entities and count how many occurrences the remaining one can have.

* Cardinality-relationship type label = determined by maximum only:

- * 1:1, 1:*, *: * } Binary Relationship only
- * one to one, one to many, many to many

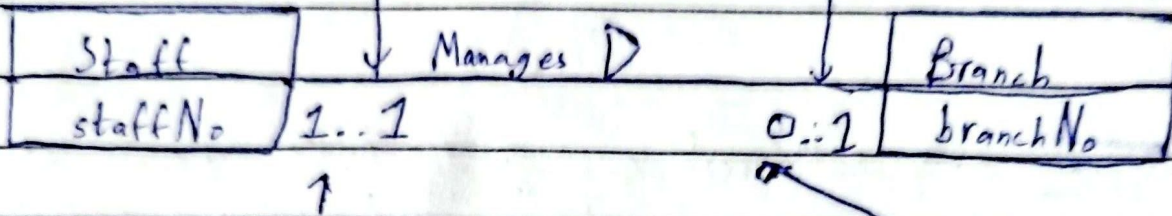
* Participation describes whether participation is optional or mandatory

- * minimum 0 → optional (some occurrences may not participate)
- * minimum 1 → mandatory (all occurrences must participate)

Date: _____

One branch is managed by one staff (Cardinality)

One member of staff manages one branch (Cardinality)



All branches are managed (Participation)

Not all staff manage branches (Optional)